

SOLID PRINCIPLES

in Object-Oriented Programming

A Practical Course in C++ Architecture by Sergino Bradford
GOUNOUKPEROU

S	Single Responsibility Principle
O	Open/Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

Language: C++17 • Level: Intermediate

Introduction & Course Overview

What are the SOLID principles?

The SOLID principles are five foundational design guidelines for writing clean, maintainable, and scalable object-oriented code. They were first assembled and popularized by Robert C. Martin ("Uncle Bob") in the early 2000s. The acronym was later coined by Michael Feathers. While the principles themselves are language-agnostic, this course uses C++ to illustrate them with concrete, real-world scenarios.

The key insight behind SOLID is this: bad software doesn't usually start bad. It becomes bad over time as requirements change and developers make small, pragmatic shortcuts. SOLID principles are a set of disciplines that, when followed consistently, keep codebases from rotting as they grow.

Course Schedule

1	Introduction & Context
2	S — Single Responsibility Principle (20 min)
3	O — Open/Closed Principle (20 min)
4	L — Liskov Substitution Principle (15 min)
5	I — Interface Segregation Principle (13 min)
6	D — Dependency Inversion Principle (12 min)

Why Learn These Principles?

Before writing a single line of code, consider these real-world pain points that SOLID directly addresses:

- You modify a class to fix a bug, and three other seemingly unrelated things break.
- Adding a new feature requires editing 10 existing files.
- Writing unit tests is near impossible because every class depends on everything.
- New team members take weeks to understand even simple components.

SOLID is the medicine for all of these. Let's begin.

S Single Responsibility Principle

Definition

"A class should have one, and only one, reason to change." — Robert C. Martin

More practically: a class should do one thing, and do it well. 'Responsibility' maps to a reason to change — so if a class would need modification for two different reasons (e.g., a database schema change AND a UI change), it has too many responsibilities.

The Problem: God Classes

A 'God Class' knows too much and does too much. This is the most common violation of SRP in production code. Below is a realistic example: a class that handles user account logic, serialization, email notifications, and logging all at once.

✗ BAD – Violates principle

```
// A UserManager that does EVERYTHING
class UserManager {
public:
    void createUser(const std::string& name, const std::string& email);
    void deleteUser(int userId);

    // Persistence
    void saveToDatabase(const User& user);
    User loadFromDatabase(int userId);

    // Email
    void sendWelcomeEmail(const User& user);
    void sendPasswordResetEmail(const User& user);

    // Serialization
    std::string serializeToJSON(const User& user);
    User deserializeFromJSON(const std::string& json);

    // Logging
    void logActivity(const std::string& activity);
};
```

This class has at least four reasons to change: the user business rules, the database schema, the email template, and the serialization format. Changing the JSON schema forces a recompile and retest of email sending. This is fragile, unstable, and hard to reuse.

The Solution: Separate Concerns

✓ GOOD – Follows principle

```
// Each class has exactly ONE reason to change
```

```

class UserRepository {
public:
    void save(const User& user);
    User findById(int userId);
};

class UserNotifier {
public:
    void sendWelcomeEmail(const User& user);
    void sendPasswordResetEmail(const User& user);
};

class UserSerializer {
public:
    std::string toJSON(const User& user) const;
    User fromJSON(const std::string& json) const;
};

class UserService {
    UserRepository    repo_;
    UserNotifier      notifier_;
public:
    void createUser(const std::string& name, const std::string& email) {
        User u{name, email};
        repo_.save(u);
        notifier_.sendWelcomeEmail(u);
    }
};

```

💡 Now, changing the email provider only touches *UserNotifier*. Switching from JSON to Protocol Buffers only touches *UserSerializer*. Each class can be unit-tested in complete isolation.

Real-World Scenario: Game Engine Component

Consider a Sprite class in a game engine. It might be tempting to let it handle its own rendering, collision, and serialization. With SRP, these are separated into a *Renderer*, a *Collider*, and a *SceneSerializer* — allowing the rendering backend to be swapped (e.g., OpenGL to Vulkan) without touching gameplay logic.

Key Takeaways — SRP

- If you struggle to name a class in one short noun phrase, it probably violates SRP.
- Ask: 'For what different stakeholders would this class change?' Each stakeholder = one class.
- SRP makes classes smaller, easier to test, and easier to reason about.

O Open/Closed Principle

Definition

"Software entities should be open for extension, but closed for modification." — Bertrand Meyer / Robert C. Martin

Once a class is written, tested, and deployed, you should be able to add new behavior without changing its existing code. You extend by adding new code (subclasses, strategy objects, lambdas), not by editing existing code. This prevents regression bugs.

The Problem: Perpetual if/else Chains

This is the most common OCP violation. You have a function or class that uses a switch or if/else block to dispatch behavior based on a type. Every new type requires modifying this class — opening it for change.

✗ BAD – Violates principle

```
enum class ShapeType { Circle, Rectangle, Triangle };

class AreaCalculator {
public:
    double calculate(ShapeType type, double a, double b) {
        if (type == ShapeType::Circle)
            return 3.14159 * a * a;
        else if (type == ShapeType::Rectangle)
            return a * b;
        else if (type == ShapeType::Triangle)
            return 0.5 * a * b;
        // Adding a Hexagon? Edit this class!
        return 0.0;
    }
};
```

The Solution: Polymorphic Extension

Introduce an abstraction. New shapes are added by writing new classes, not modifying existing ones. The AreaCalculator never needs to change again.

✓ GOOD – Follows principle

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius_;
```

```

public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
};

class Rectangle : public Shape {
    double w_, h_;
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
};

// Adding a Hexagon? Just add this class – nothing else changes.
class Hexagon : public Shape {
    double side_;
public:
    explicit Hexagon(double s) : side_(s) {}
    double area() const override { return (3.0 * std::sqrt(3.0) / 2.0) * side_ *
side_; }
};

class AreaCalculator {
public:
    // CLOSED for modification – will never change again
    double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) {
        double total = 0.0;
        for (const auto& s : shapes) total += s->area();
        return total;
    }
};

```

Modern C++: OCP with `std::variant` and `std::visit`

In modern C++17, you can also achieve OCP without inheritance using `std::variant` and `std::visit`, which avoids virtual dispatch overhead. This is especially popular in performance-sensitive code (game engines, parsers):

```

using ShapeVariant = std::variant<Circle, Rectangle, Hexagon>;

struct AreaVisitor {
    double operator()(const Circle& c)    const { return 3.14159 * c.radius *
c.radius; }
    double operator()(const Rectangle& r) const { return r.w * r.h; }
    double operator()(const Hexagon& h)   const { return
(3.0*std::sqrt(3.0)/2.0)*h.side*h.side; }
};

double computeArea(const ShapeVariant& shape) {
    return std::visit(AreaVisitor{}, shape);
}

```

 The visitor pattern with `std::variant` is closed for modification of the computation logic, and open for extension by adding new overloads or visitors.

Real-World Scenario: Payment Processing

A payment service that started with CreditCard payments now needs PayPal, Stripe, and Crypto. With OCP, each payment method is an independent class implementing a `PaymentProcessor` interface. The order fulfillment code is never touched — it calls `process()` polymorphically. No regression risk.

Key Takeaways — OCP

- Design for the changes you anticipate. Don't over-engineer, but design 'seams' where extension points are likely.
- Prefer adding new code over editing existing, tested code.
- In C++, achieve OCP via virtual dispatch, templates, or `std::variant/visit`.

Liskov Substitution Principle

Definition

"If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program." — Barbara Liskov, 1987

In simpler terms: a derived class must be usable anywhere its base class is used, without the caller needing to know the difference. Violating LSP means your inheritance hierarchy is lying to callers about what they can expect.

The Classic Violation: Square-Rectangle Problem

This is the textbook example of an LSP violation. A Square IS-A Rectangle mathematically, but in OOP, inheriting Square from Rectangle breaks behavioral contracts:

✗ BAD – Violates principle

```
class Rectangle {
public:
    virtual void setWidth(double w) { width_ = w; }
    virtual void setHeight(double h) { height_ = h; }
    double area() const { return width_ * height_; }
protected:
    double width_ = 0, height_ = 0;
};

class Square : public Rectangle {
public:
    // Must keep sides equal – breaks Rectangle's contract!
    void setWidth(double w) override { width_ = height_ = w; }
    void setHeight(double h) override { width_ = height_ = h; }
};

// This function works with Rectangle but BREAKS with Square
void testRectangle(Rectangle& r) {
    r.setWidth(5.0);
    r.setHeight(4.0);
    // Expected: 20.0 | With Square: 16.0 – SURPRISE!
    assert(r.area() == 20.0); // Fails for Square!
}
```

The fix here is not to force the Square-Rectangle inheritance. Both should independently implement a Shape interface, or Square should simply not inherit from Rectangle.

A Real-World Violation: ReadOnly Collection

A common LSP pitfall in production code is inheriting from a mutable collection to create a read-only version:

✗ BAD – Violates principle

```
class MutableList {
public:
    virtual void add(int item) { items_.push_back(item); }
    virtual void remove(int item) { /* ... */ }
    int get(int index) const { return items_[index]; }
    size_t size() const { return items_.size(); }
private:
    std::vector<int> items_;
};

class ReadOnlyList : public MutableList {
public:
    void add(int) override { throw std::logic_error("Read only!"); }
    void remove(int) override { throw std::logic_error("Read only!"); }
};

// Caller trusts MutableList contract – gets a runtime bomb:
void populateList(MutableList& list) {
    list.add(42); // Throws if passed a ReadOnlyList – LSP violated!
}
```

✓ GOOD – Follows principle

```
// GOOD: Separate the read and write contracts
class ReadableList {
public:
    virtual int get(int index) const = 0;
    virtual size_t size() const = 0;
    virtual ~ReadableList() = default;
};

class MutableList : public ReadableList {
public:
    virtual void add(int item) = 0;
    virtual void remove(int item) = 0;
};

// ReadOnlyList implements only ReadableList – no broken promises
class ReadOnlyList : public ReadableList { /* ... */ };

// This function clearly advertises it needs a mutable list
void populateList(MutableList& list) {
    list.add(42); // Contract is honored by all MutableList subtypes
}
```

LSP and Contracts

The formal way to think about LSP is in terms of preconditions, postconditions, and invariants. A subclass must not strengthen preconditions (it can't demand MORE from callers), must not weaken postconditions (it can't guarantee LESS to callers), and must preserve class invariants.

💡 *Prefer composition over inheritance when you cannot guarantee behavioral substitutability. Not every IS-A relationship in language translates to a safe IS-A in code.*

Key Takeaways — LSP

- If a subclass throws exceptions the base class doesn't, or overrides methods with no-ops, it's an LSP violation.
- Design inheritance around behavior, not taxonomy. 'Is-a' in English doesn't always mean 'is-a' in OOP.
- Use the Tell, Don't Ask principle: callers should trust the interface, not check the concrete type.

I Interface Segregation Principle

Definition

"No client should be forced to depend on methods it does not use." — Robert C. Martin

Fat interfaces create unwanted coupling. When a class implements a large interface and only needs a few of its methods, it is forced to provide stub implementations for the rest. Changes to the unused parts of the interface ripple into unrelated classes.

The Problem: The Fat Interface

Imagine a document management system with a single Printable interface that mixes printing, scanning, faxing, and stapling. A simple PDF exporter doesn't fax — but it's forced to depend on the fax method:

✗ BAD – Violates principle

```
// Fat interface – forces all implementors to know about everything
class IMultifunctionDevice {
public:
    virtual void print(Document& doc) = 0;
    virtual void scan(Document& doc) = 0;
    virtual void fax(Document& doc) = 0;
    virtual void staple(Document& doc) = 0;
    virtual ~IMultifunctionDevice() = default;
};

// A basic printer – must stub out scan, fax, staple:
class BasicPrinter : public IMultifunctionDevice {
public:
    void print(Document& doc) override { /* real implementation */ }
    void scan(Document& doc) override { /* ??? not supported */ }
    void fax(Document& doc) override { /* ??? not supported */ }
    void staple(Document& doc) override { /* ??? not supported */ }
};
```

The Solution: Segregated Interfaces

✓ GOOD – Follows principle

```
class IPrinter {
public:
    virtual void print(Document& doc) = 0;
    virtual ~IPrinter() = default;
};

class IScanner {
public:
    virtual void scan(Document& doc) = 0;
    virtual ~IScanner() = default;
};
```

```

};

class IFax {
public:
    virtual void fax(Document& doc) = 0;
    virtual ~IFax() = default;
};

// Basic printer implements ONLY what it supports
class BasicPrinter : public IPrinter {
public:
    void print(Document& doc) override { /* implementation */ }
};

// A high-end device can implement multiple interfaces
class OfficeDevice : public IPrinter, public IScanner, public IFax {
public:
    void print(Document& doc) override { /* ... */ }
    void scan(Document& doc) override { /* ... */ }
    void fax(Document& doc) override { /* ... */ }
};

// Callers only depend on what they need:
void printReport(IPrinter& printer, Document& doc) {
    printer.print(doc); // Doesn't care about fax or scan
}

```

ISP and Template Parameters in C++

In modern C++, ISP also applies to template constraints. With C++20 concepts, you can express minimal interface requirements explicitly, without needing a virtual interface at all:

```

// C++20 concept – minimal interface contract
template<typename T>
concept Printable = requires(T t, Document d) {
    { t.print(d) } -> std::same_as<void>;
};

// Only demands 'print' – nothing else
void printAll(Printable auto& device, Document& doc) {
    device.print(doc);
}

```

 *ISP naturally leads to the principle of 'program to the narrowest interface you need.' This makes code more flexible and easier to mock in tests.*

Key Takeaways — ISP

- If a class implementing an interface has empty or throw-based stub methods, the interface is too fat.
- Small, focused interfaces are easier to mock, test, and implement.

- In C++, use pure abstract classes for polymorphic interfaces, or concepts for static (template) interfaces.

D Dependency Inversion Principle

Definition

"High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions." — Robert C. Martin

DIP is about the direction of your dependencies. High-level policy code (e.g., business logic) should not reach down and directly instantiate low-level code (e.g., database drivers, HTTP libraries). Both should depend on an interface that neither owns. This is what makes code testable and swappable.

The Problem: Direct Low-Level Dependency

A high-level OrderService that directly depends on a concrete MySQLDatabase class is tightly coupled. You cannot unit test OrderService without a live database. You cannot switch to PostgreSQL without rewriting OrderService.

✗ BAD – Violates principle

```
// Low-level detail
class MySQLDatabase {
public:
    void saveOrder(const Order& order) {
        // Executes SQL INSERT against MySQL...
    }
};

// High-level policy – DIRECTLY depends on low-level detail
class OrderService {
    MySQLDatabase db_; // <-- hardwired dependency!
public:
    void placeOrder(const Order& order) {
        // ... business logic ...
        db_.saveOrder(order); // Can't test without MySQL!
    }
};
```

The Solution: Depend on Abstractions (Dependency Injection)

✓ GOOD – Follows principle

```
// The abstraction – owned by the high-level module
class IOrderRepository {
public:
    virtual void save(const Order& order) = 0;
    virtual ~IOrderRepository() = default;
};
```

```

// Low-level detail implements the abstraction
class MySQLOrderRepository : public IOrderRepository {
public:
    void save(const Order& order) override {
        // MySQL-specific implementation
    }
};

// For testing – a fast, in-memory fake
class InMemoryOrderRepository : public IOrderRepository {
    std::vector<Order> orders_;
public:
    void save(const Order& order) override {
        orders_.push_back(order);
    }
};

// High-level module depends ONLY on the abstraction
class OrderService {
    IOrderRepository& repo_; // Injected – not created here
public:
    explicit OrderService(IOrderRepository& repo) : repo_(repo) {}

    void placeOrder(const Order& order) {
        // ... business logic ...
        repo_.save(order); // Works with MySQL, Postgres, or FakeRepo
    }
};

// Production setup:
MySQLOrderRepository prodRepo;
OrderService prodService(prodRepo);

// Test setup – no database needed!
InMemoryOrderRepository testRepo;
OrderService testService(testRepo);

```

Forms of Dependency Injection in C++

There are three standard ways to inject dependencies in C++: constructor injection (shown above, preferred for required dependencies), setter injection (for optional dependencies), and interface injection (rare). Constructor injection is generally recommended because it makes all dependencies explicit and the object is always in a valid state after construction.

 *DIP is the foundation of testability. If you can't inject a dependency, you can't replace it with a test double (mock/stub/fake). A class that creates its own dependencies is impossible to unit test in isolation.*

DIP and std::function / Lambdas

For simple dependencies, you don't even need a full interface class. A std::function is a lightweight abstraction over any callable:

```
class OrderService {
```

```

    std::function<void(const Order&)> save_;
public:
    explicit OrderService(std::function<void(const Order&)> save)
        : save_(std::move(save)) {}

    void placeOrder(const Order& order) {
        save_(order); // Works with any callable – lambda, functor, function ptr
    }
};

// Inject a lambda in tests:
std::vector<Order> captured;
OrderService svc([&](const Order& o){ captured.push_back(o); });

```

Key Takeaways — DIP

- If you see 'new SomeConcreteThing()' inside a high-level class, that's likely a DIP violation.
- Inject dependencies through constructors. The caller decides the implementation; the class doesn't.
- DIP is what makes unit testing possible and makes your architecture swap-friendly.

Conclusion & The Bigger Picture

How the Principles Work Together

The five SOLID principles are not independent. They reinforce each other and, taken together, push you towards a specific architectural style:

Principle	Core Question	Primary Benefit
SRP	<i>Should this class exist?</i>	Reduces side effects when changing one thing
OCP	<i>Can I add without modifying?</i>	Prevents regression bugs in existing code
LSP	<i>Is my inheritance honest?</i>	Makes polymorphism safe and predictable
ISP	<i>Is my interface too broad?</i>	Reduces coupling, simplifies testing
DIP	<i>Who controls the dependency?</i>	Enables testability and replaceability

Common Mistakes to Avoid

- Over-engineering: Applying SOLID to tiny scripts or trivial functions adds needless complexity. SOLID is for code that will grow and change.
- Interface explosion: Creating an interface for every single class is ISP taken too far. Introduce abstractions when you have multiple implementations or want to decouple for testing.
- Applying LSP too rigidly: Stateless utility subclasses or pure extensions often don't need full behavioral substitutability.

Going Further

SOLID is a starting point, not a finish line. These principles pair naturally with Design Patterns (Gang of Four), Clean Architecture, and Test-Driven Development. Reading recommendations: Clean Code and Clean Architecture by Robert C. Martin, and Effective C++ / Modern Effective C++ by Scott Meyers for the C++-specific implementation nuances.

— *End of Course* —